UI부터 Supabase 커밋까지

문제 100개가 어떻게 안전하게 두 개의 세트가 되는가

SQLite · Python · Psycopg 3 · PostgreSQL · Supabase

> 처음 배우는 사람을 위한 실제 코드 해설서
> 기초 개념에서 시작해 UI 버튼, SQL, 커밋, 실패 복구까지 연결합니다.

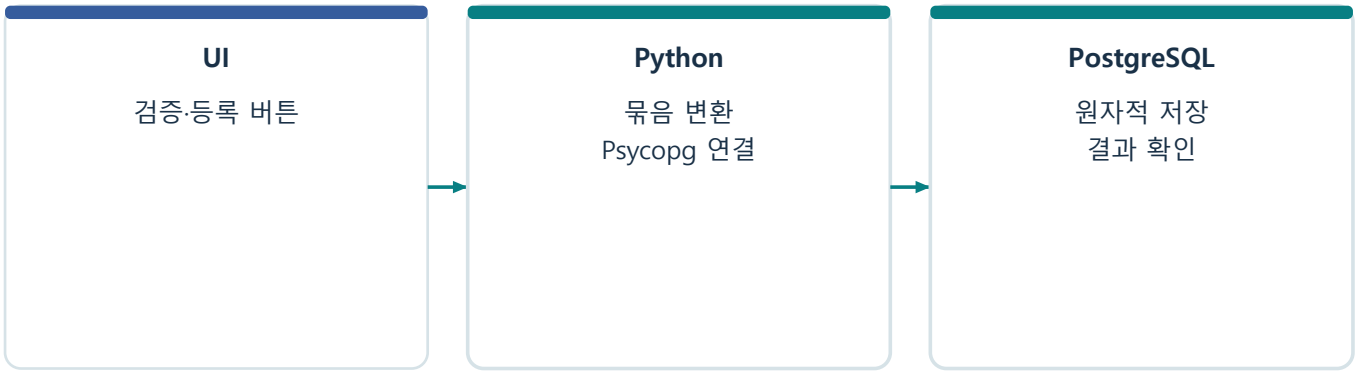


그림 1. 한 번의 등록을 세 가지 층으로 나누어 이해한다.

이 책에서 끝까지 따라갈 사례

DeepSeek 4.1로 생성한 읽기 50문항과 듣기 50문항을 함께 선택해 Supabase 문항 은행에 등록한 실제 작업입니다. 기존 원격 데이터 위에 2세트·100문항을 추가했습니다.

코드 기준일: 2026-10-05
Psycopg: 3.3.4 / binary 구현
원격 PostgreSQL: 17.6 / Session pooler: 5432

읽는 순서와 목차

먼저 3~7쪽으로 용어와 데이터 구조를 이해한 뒤, 8~16쪽에서 UI와 코드를 따라가세요. 17~24쪽은 안전장치와 장애 복구, 25~28쪽은 실제 결과와 찾아보기입니다.

03~06쪽 전체 구조·DB 기초·Psycopg·매개변수

07~10쪽 테이블 관계·UI·검증 패키지·식별자

11~14쪽 연결·with·실제 발행·SQL 실행 순서

15~16쪽 문항 버전과 세트 연결 INSERT

17~20쪽 조회 가시성·잠금·WAL·사후 검증

21~24쪽 롤백·통신 단절·로컬 상태·재시도

25~28쪽 실제 결과·파일 역할·용어·참고문헌

표시를 구분해서 읽으세요

표시	뜻
실제 코드 발췌	현재 저장소에서 가져온 코드입니다. 원래 파일 줄 번호를 표시합니다.
학습용 예제	작동 원리를 설명하기 위해 만든 작은 예제입니다. 앱에서 실행한 코드가 아닙니다.
실제 실행 기록	이번 100문항 등록의 기록과 확인 결과입니다.
UI 재구성	실제 버튼명과 코드 흐름을 그린 설명용 그림입니다. 화면 캡처가 아닙니다.

> 가장 중요한 구분: SQL 실행 성공 ≠ 원격 COMMIT 확인 ≠ 로컬 등록 완료 ≠ 학습자 공개.

전체 구조: 누가 무엇을 하는가

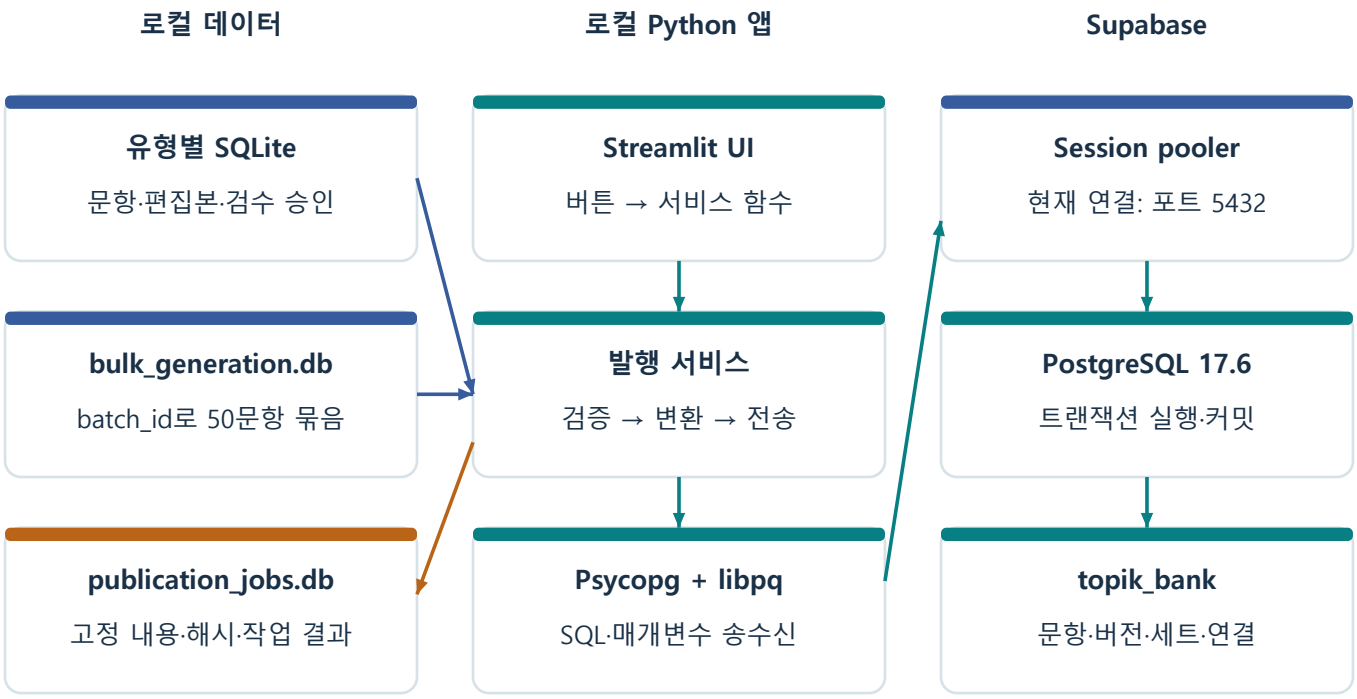


그림 2. 데이터는 로컬에서 변환되며, 원격 SQL은 Psycopg/libpq와 Session pooler를 거쳐 실행된다.

Streamlit은 버튼과 목록을 보여주는 UI입니다. 버튼을 누르면 로컬 Python 서비스 함수가 실행됩니다. SQLite는 생성·편집·승인 원본을 보관하고, 발행 서비스는 해당 묶음을 원격 테이블 형식으로 바꿉니다.

Psycopg는 Python이 PostgreSQL과 대화하게 하는 드라이버입니다. 실제 쿼리 실행과 제약 검사, 커밋·롤백은 Supabase 안의 PostgreSQL이 담당합니다. Pooler는 앱 연결과 서버 연결을 중개합니다.

이번 경로에서 쓰는 것	역할
PRODUCTION_DATABASE_URL	Supabase PostgreSQL 연결 정보. 로컬 환경변수에서 읽습니다.
publication_jobs.db	검증한 내용과 작업 상태를 로컬에 기록합니다.
topik_bank 스키마	원격 문항·버전·세트와 순서를 저장합니다.

근거: 현재 코드 · Supabase 연결 문서 [4]

DB와 SQL, 연결과 트랜잭션

용어	쉬운 설명	이번 앱의 예
데이터베이스(DB)	표와 데이터를 관리하는 시스템·저장 공간	SQLite 원본, Supabase PostgreSQL
테이블 / 행	같은 형식의 기록을 모은 표 / 기록 하나	items / 문항 하나
SQL	DB에 조회·저장을 요청하는 언어	SELECT, INSERT
드라이버	Python 요청을 DB가 이해하는 형태로 전달	Psycopg 3
연결(Connection)	Python에서 DB와 대화하는 통로를 나타내는 객체	변수 c
세션(Session)	접속 중 DB가 유지하는 대화 상태	설정·진행 중 트랜잭션
커서(Cursor)	SQL 실행 결과를 다루는 객체	execute()가 돌려주는 결과
트랜잭션	여러 작업을 함께 확정하거나 취소하는 단위	두 세트를 한 번에 저장

SQL 명령 하나와 트랜잭션 하나는 다릅니다

문항 100개를 저장하려면 많은 INSERT와 SELECT가 필요합니다. 이 SQL들이 같은 연결의 같은 트랜잭션 안에서 실행되면, 앞부분만 저장되고 뒷부분이 빠지는 상태로 확정되는 것을 막을 수 있습니다.

> 가게에서 주문서, 주문 상품, 결제 내역을 함께 확정하는 것을 떠올려 보세요. 여기서는 문항, 문항 버전, 세트, 세트의 문항 순서가 함께 갖춰져야 합니다.

근거: Psycopg 기본 사용법 [2] · 프로젝트 코드

Psycopg는 무엇이고 어떻게 작동할까

Psycopg는 PostgreSQL용 Python 드라이버입니다. 현재 프로젝트는 `psycopg[binary]>=3.2,<4`를 요구하며, 설치된 버전은 3.3.4입니다. binary 구현은 C 기반 구성 요소와 libpq를 함께 사용합니다.

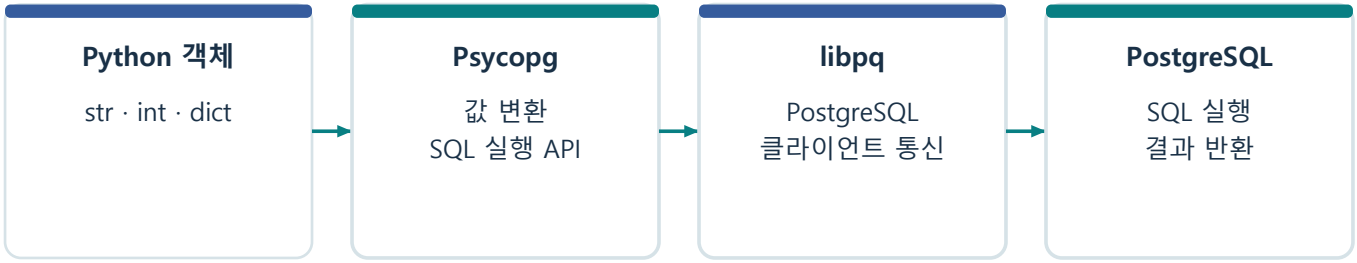


그림 3. 드라이버는 값과 결과를 변환하고, DB 서버는 실제 저장과 트랜잭션을 수행한다.

코드 1 | 학습용 예제 · 연결 → 쿼리 → 결과 읽기

UI: 직접 화면 없음 / 입력: 두 정수 / 출력: 50

```python
import os
import psycopg

dsn = os.environ["PRODUCTION_DATABASE_URL"]
with psycopg.connect(dsn) as conn:
    result = conn.execute(
        "SELECT %s::int + %s::int",
        (20, 30),
    )
    number = result.fetchone()[0]
    # number == 50
```

`connect()`는 연결 객체를 돌려줍니다. `execute()`는 SQL과 값을 보내고 커서를 돌려줍니다. `fetchone()`은 한 행을 Python 튜플로 가져오므로 `[0]`으로 첫 열을 읽습니다.

여러 커서를 만들어도 같은 연결을 사용하면 같은 트랜잭션에 속합니다. 연결을 새로 만들면 별도 세션과 트랜잭션이므로, 미커밋 상태를 보는 방식도 달라집니다.

dsn은 접속 정보를 담은 문자열입니다. 예제는 환경변수에서 읽으며 실제 주소·인증값은 적지 않았습니다. 환경변수는 이 Python 실행 전에 설정되어 있다고 가정합니다.

근거: Psycopg 기본 사용법 [2] · libpq 모듈 [5]

값은 어떻게 SQL로 전달되는가

코드 2 | 학습용 예제 · SQL 문장과 데이터 값을 분리

UI: 직접 화면 없음 / 입력: 정답·dict / 출력: DB 행

```python
from psycopg.types.json import Jsonb

# conn과 example 테이블이 이미 준비됐다고 가정합니다.
conn.execute(
    "INSERT INTO example(answer, content) VALUES (%s, %s)",
    (1, Jsonb({"choices": ["가", "나", "다", "라"]})),
)
```

`%s`는 Python 문자열 치환 표시가 아니라 Psycopg의 값 자리표시자입니다. SQL과 값 튜플을 별도로 전달하면 드라이버가 값을 적절한 DB 자료형으로 바꿉니다. 문자열의 따옴표를 직접 이어 붙일 필요가 없습니다.

`Jsonb(dict)`는 Python 딕셔너리를 PostgreSQL JSONB로 전달합니다. 문항 전체 구조인 `content_json`, 생성 출처인 `source_provenance`, 보기 목록 등이 이 방식으로 저장됩니다.

정수도 문자열도 값 자리에는 `%s`를 쓰고 따옴표로 감싸지 않습니다. 값이 하나일 때의 `(1,)`는 한 원소 튜플입니다. 끝의 쉼표가 없는 `(1)`은 튜플이 아닙니다.

메서드·자료형	이번 코드에서 하는 일
execute(sql, values)	문항·세트 저장과 충돌·버전 조회
fetchone() / fetchall()	조회 결과를 한 행 / 여러 행으로 읽음
Jsonb(...)	문제 dict와 보기 list를 JSONB로 변환
cursor.executemany(sql, rows)	같은 INSERT 틀로 1~50번 연결 행을 반복 저장

> executemany()는 반복 실행 도구입니다. 전체 발행의 원자성은 같은 연결·트랜잭션과 COMMIT/ROLLBACK 경계에서 나옵니다. 네트워크 전송 횟수와 트랜잭션 개수는 같지 않습니다.

근거: Psycopg 매개변수 [6] · 자료형 변환 [7] · 기본 사용법 [2]

한 세트는 네 개의 표로 표현된다



그림 4. Q1과 S읽기는 관계를 보여주기 위한 기호이며 실제 UUID가 아니다. 화살표는 참조 방향이다.

`items`는 문항의 지속적인 신원입니다. `item_versions`는 그 문항의 특정 내용을 보관합니다. 같은 문항이 수정되면 버전이 달라질 수 있기 때문에 두 개념을 나눕니다.

`question_sets`는 읽기 또는 듣기 세트 자체입니다. `question_set_items`의 각 행은 세트의 한 번호를 `(item_id, item_version)`에 연결합니다. 따라서 1번은 문항의 막연한 최신 내용이 아니라 지정된 버전을 읽습니다.

한 행의 예	의미
S읽기 / position=1 / Q1 / version=1	읽기 세트의 1번은 Q1 문항의 1번 버전
S읽기 / position=2 / Q2 / version=1	같은 세트의 2번은 다른 문항 Q2
S듣기 / position=1 / L1 / version=1	듣기는 별도의 세트·문항 연결

UI 버튼이 실제 함수로 이어지는 경로

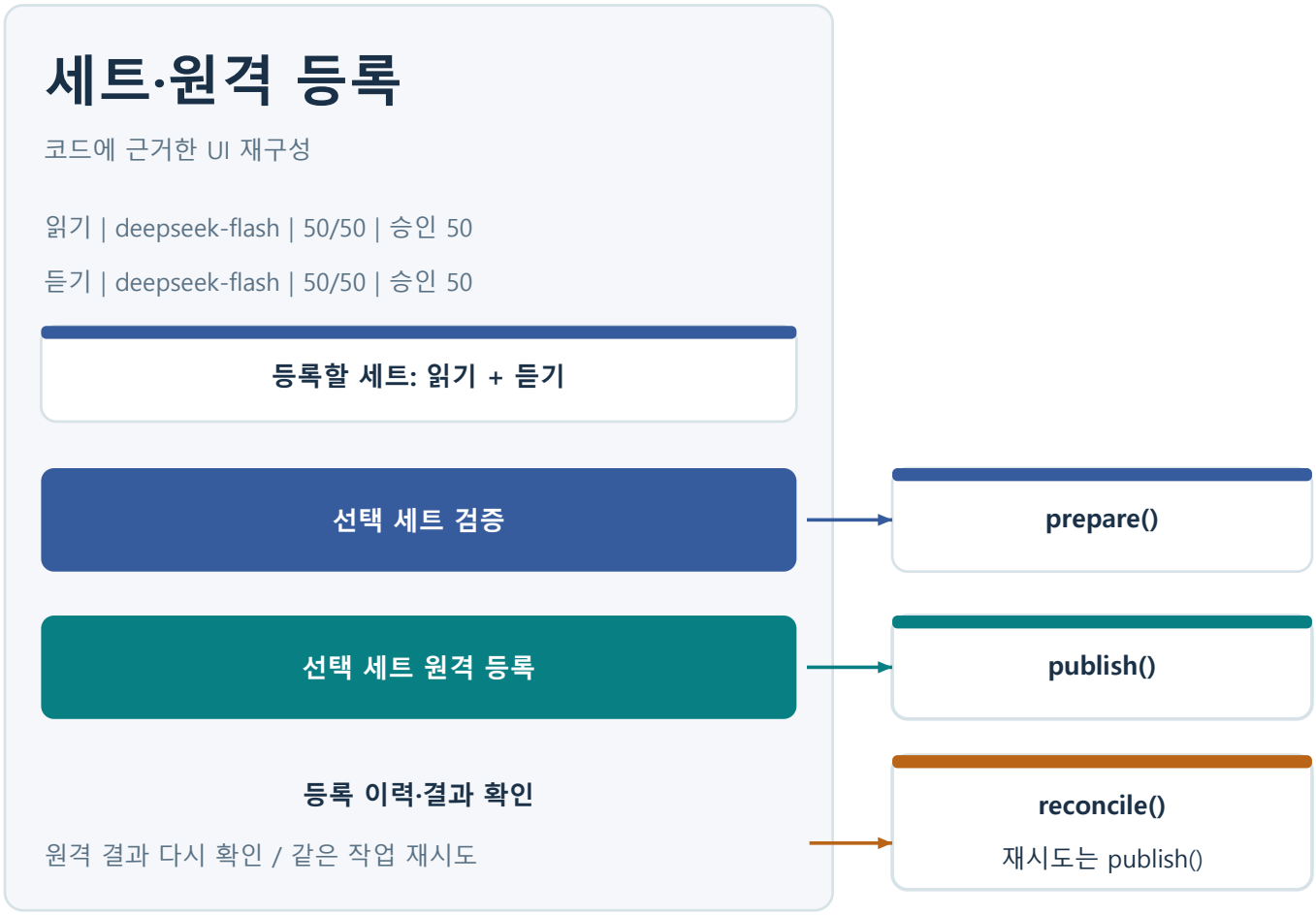


그림 5. 앱 코드의 버튼명·서비스 호출을 재구성했다. 실제 브라우저 캡처가 아니다.

코드 3 | 실제 코드 발췌 · Streamlit 버튼 이벤트의 서비스 호출 (중간 생략)

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publish_app.py

UI: 그림의 네 버튼 / 검증: 묶음·설정 → 작업 / 등록·조회·재시도: 작업 ID → 결과

```
71  job = service.prepare(selected, metadata)
    ...  # 중간 구간 생략
85      result = service.publish(plan_id)
    ...  # 중간 구간 생략
111 service.reconcile(job["id"])
    ...  # 중간 구간 생략
118     service.publish(job["id"])
```

검증과 결과 확인은 원격 읽기 전용 조회입니다. 원격 등록·재시도만 필요한 INSERT를 수행합니다. 검증 뒤의 작업 ID가 다음 버튼의 입력으로 이어집니다.

버튼 처리 위치: 검증 68~77행, 등록 82~94행, 결과 확인 109~114행, 재시도 116~121행. 메뉴 등록은 main.py, 생성 완료 화면의 바로가기는 bulk_generation_app.py가 담당합니다.

등록 전에 검증한 내용을 고정한다

입력은 선택한 생성 묶음 ID와 급수·난이도·Skill 설정입니다. 기본 급수는 4, 예상 난이도는 0.0이며 Skill은 유형별 기본값을 씁니다. 각 묶음의 현재 50문항을 읽고 승인·정답·보기·공통 대본 등을 검사해 전송 패키지를 만듭니다.

코드 4 | 실제 코드 발췌 · DirectPublicationService.prepare()

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 선택 세트 검증 / 입력: batch_ids, metadata / 출력: 저장된 작업

```
354 def prepare(self, batch_ids, metadata=None):
355     if not batch_ids:
356         raise PublicationBlocked("세트를 선택하세요.")
357     catalog = self.catalog()
358     batches = {b["id"]: b for b in read_batches(self.root)}
359     packages = []
360     for batch_id in sorted(set(batch_ids)):
361         if batch_id not in batches:
362             raise PublicationBlocked("선택한 생성 묶음이 없습니다.")
363         packages.append(make_package(batches[batch_id], [r for r in catalog.records if
      r.candidate.batch_id == batch_id], metadata))
364     try:
365         with self.connect(readonly=True) as c:
366             c.execute("SET TRANSACTION ISOLATION LEVEL REPEATABLE READ")
367             require_target_schema(c)
368             for p in packages:
369                 check_remote(c, p)
370     except Exception as exc:
371         raise PublicationBlocked(_safe_error(exc, self.target_url)) from exc
372     return self.jobs.save(self.target, packages)
```

366~370행은 원격의 스키마와 기존 내용을 읽어 호환성·충돌을 확인합니다. 372행에서는 검증한 후보·메타데이터·해시를 로컬 작업 DB에 저장합니다. 원격 등록 트랜잭션은 아직 시작하지 않았습니다.

> 검증 완료는 "보낼 내용이 준비됐음"입니다. 이 단계에서 Supabase에 문항 100개가 추가된 것은 아닙니다.

묶음·문항·작업을 다시 알아보는 방법

코드 5 | 실제 코드 발췌 · _json(), _hash(), batch_set_id()

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 검증·재시도 / 입력: 내용 또는 묶음 / 출력: SHA-256·UUIDv5

```
35  def _json(value):
36      return json.dumps(value, ensure_ascii=False, sort_keys=True, separators=(",", ":"),
      allow_nan=False)
37
38
39  def _hash(value):
40      return hashlib.sha256(_json(value).encode()).hexdigest()
...   # 중간 구간 생략
66  def batch_set_id(section, batch_id):
67      return str(uuid.uuid5(PUBLICATION_NAMESPACE, f"batch-set:{section}:{batch_id}"))
```

식별자	무엇을 구분하는가	재시도 때 쓰임
batch_id	한 번의 읽기·듣기 일괄 생성 묶음	다른 생성 결과를 섞지 않음
source_key / item_id	로컬의 특정 문항과 원격 문항 신원	같은 원본인지 식별
set_id	영역+batch_id에서 만든 UUIDv5	같은 묶음은 같은 세트 ID
source_hash / content_hash	검증한 원본 / 변환된 문항 내용	수정·내용 불일치 감지
plan_id	대상 DB+전체 전송 패키지의 해시	저장된 같은 작업을 복구

UUIDv5는 이름에서 일정한 UUID를 만드는 방법입니다. SHA-256은 내용이 같은지 비교하는 지문입니다. 둘 다 이름·내용을 판별하는 도구이며, 저장 자체를 원자적으로 만드는 장치는 아닙니다.

코드는 `_check_current()`에서 현재 로컬 문항으로 패키지를 다시 만들어 고정 패키지와 비교합니다. 등록 직전과 원격 잠금 획득 후에 재검사합니다. 이 검사는 SQLite 편집 자체를 잠그는 기능은 아닙니다.

Supabase 연결 객체는 어떻게 만들어질까

코드 6 | 실제 코드 발췌 · DirectPublicationService.connect()

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 검증·등록·결과 확인 / 입력: readonly / 출력: Psycopg Connection

```
314 def connect(self, readonly=False):
315     if not self.target_url:
316         raise PublicationBlocked("PRODUCTION_DATABASE_URL이 설정되지 않았습니다.")
317     psycopg, _ = _psycopg()
318     return psycopg.connect(self.target_url, connect_timeout=8, prepare_threshold=None,
319         options=f"-c default_transaction_read_only={'on' if readonly else 'off'} -c
  statement_timeout=60000 -c lock_timeout=15000")
```

설정	이번 코드의 의미
PRODUCTION_DATABASE_URL	호스트·DB·사용자·인증 정보를 담은 로컬 설정
connect_timeout=8	연결을 시작할 때의 제한: 8초
statement_timeout=60000	서버의 SQL 문장 실행 제한: 60초
lock_timeout=15000	잠금 획득 대기 제한: 15초
prepare_threshold=None	자동 named prepared statement 사용을 끔
readonly=True	조회 연결은 서버의 기본 트랜잭션을 읽기 전용으로 설정

현재 설정은 Supabase Session pooler의 5432 포트입니다. 세션 동안 서버 연결을 중개하는 방식이며, 트랜잭션마다 서버 연결을 배정하는 Transaction pooler와 구분해야 합니다.

prepare_threshold=None은 자동으로 이름을 붙여 서버에 보관하는 prepared statement를 끄는 설정입니다. %s로 SQL과 값을 분리해서 전달하는 기능은 계속 사용합니다.

> autocommit을 지정하지 않았으므로 기본값 False입니다. connect() 호출 자체나 with 진입이 곧 BEGIN을 의미하지는 않습니다. 첫 DB 명령을 실행할 때 암묵적 트랜잭션이 시작됩니다.

근거: Supabase 연결 문서 [4] · Psycopg 준비된 문장 [8]

with 한 블록에 커밋과 롤백을 맡긴다

코드 7 | 학습용 예제 · 암묵적 트랜잭션

UI: 직접 화면 없음 / 정상 종료 시 2행 확정

```
with psycopg.connect(dsn) as conn:
    # 연결은 만들어졌지만 SQL 실행 전입니다.
    conn.execute("INSERT INTO example(id) VALUES (%s)", (1,))
    # 첫 execute: 드라이버가 BEGIN을 시작하고 INSERT 실행
    conn.execute("INSERT INTO example(id) VALUES (%s)", (2,))
    # 같은 conn: 같은 트랜잭션
# 정상 종료: COMMIT 후 연결 닫기
```

코드 8 | 학습용 예제 · with를 이해하기 위한 개념적 등가 코드

UI: 직접 화면 없음 / 실제 앱의 대체 구현이 아님

```
conn = psycopg.connect(dsn)
try:
    # 실제 SQL과 검증 코드
    do_work(conn)
except BaseException:
    conn.rollback()
    raise
else:
    conn.commit()
finally:
    conn.close()
```

`with`는 예외가 블록 밖으로 전달됐는지에 따라 종료 처리를 합니다. 실제 코드에서는 DB 오류뿐 아니라 커밋 전 검증 실패도 예외를 발생시키므로 같은 롤백 경로를 탑니다.

> 원자성은 Psycopg의 with와 PostgreSQL 트랜잭션이 함께 구현합니다. with는 종료를 관리하고, PostgreSQL은 모든 변경의 확정·취소를 수행합니다.

근거: Psycopg 연결 컨텍스트 [2] · 트랜잭션 관리 [1]

실제 등록 함수의 트랜잭션 경계

코드 9 | 실제 코드 발췌 · DirectPublicationService.publish()

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 선택 세트 원격 등록·같은 작업 재시도 / 입력: plan_id / 출력: 등록 결과 또는 예외

```
406 def publish(self, plan_id):
407     job = self._job(plan_id)
408     try:
409         self._check_current(job)
410     except (ValueError, KeyError, TypeError) as exc:
411         self.jobs.update(plan_id, "stale", message=str(exc))
412         raise PublicationBlocked(str(exc)) from exc
413     if job["status"] in {"publishing", "outcome_unknown", "succeeded"}:
414         checked = self.reconcile(plan_id)
415         if checked["status"] == "succeeded":
416             return checked
417         if checked["status"] != "failed":
418             raise PublicationBlocked(checked["message"] or "원격 반영 결과를 먼저
    확인해야 합니다.")
419     self.jobs.update(plan_id, "publishing")
420     try:
421         with self.connect() as c:
422             # Same order everywhere: deployment, membership, item writer.
423             for lock in ("topik_bank:production_deployment",
    "topik_bank:set_membership", "topik_bank:item_sync"):
424                 c.execute("SELECT pg_advisory_xact_lock(hashtext(%s))", (lock,))
425             require_target_schema(c)
```

코드를 순서대로 읽어보세요

407~412행은 고정 작업을 읽고 로컬 원본 변경을 검사합니다. 413~418행은 이전 시도가 있었으면 실제 원격 상태부터 확인합니다. 이미 정확히 반영됐으면 INSERT를 반복하지 않고 결과를 돌려줍니다.

419행의 `publishing` 기록은 로컬 SQLite에서 먼저 확정됩니다. 421행부터 원격 연결 c를 사용하는 블록입니다. 424행의 첫 잠금 SQL이 실행되면 원격 트랜잭션이 시작됩니다.

> 같은 함수 안에 있다고 같은 DB 트랜잭션인 것은 아닙니다. 419행은 로컬 DB, 421행 이후의 SQL은 원격 DB에 대한 별도의 작업입니다.

여러 SQL을 하나의 저장 단위로 묶는다

코드 10 | 실제 코드 발췌 · 같은 연결 c로 두 세트 저장·검증

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 선택 세트 원격 등록 / 입력: 고정 packages / 출력: 커밋 전 complete 확인

```
    # Lock acquisition may have waited while someone edited SQLite.
    self._check_current(job)
    packages = job["payload"]["packages"]
    statuses = [check_remote(c, p) for p in packages]
    for p, status in zip(packages, statuses):
        if status["status"] == "absent":
            self._insert(c, p)
    for p in packages:
        if check_remote(c, p)["status"] != "complete":
            raise PublicationBlocked("전송 후 세트 검증에 실패했습니다.")
return self.reconcile(plan_id)
```

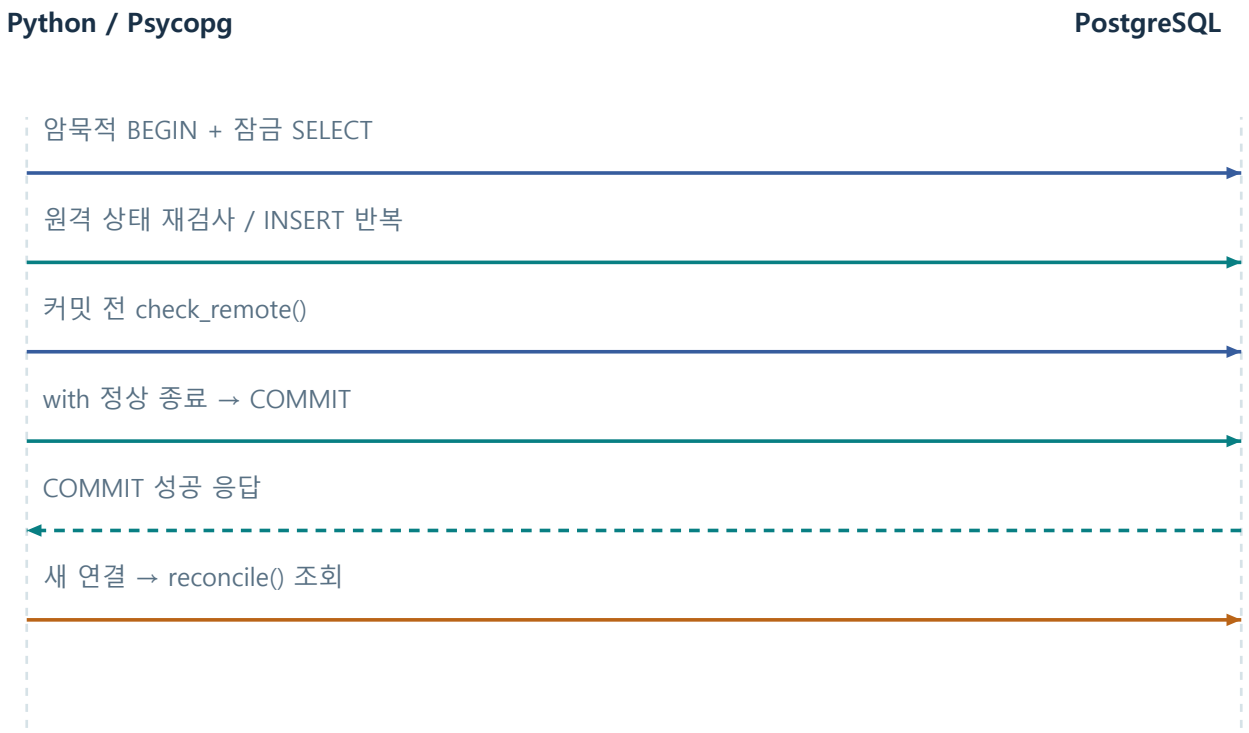


그림 6. 여러 SQL 왕복이 있어도 원격 저장은 한 트랜잭션이다. 실제 연결 중개 경로는 3쪽에 표시했다.

432행 `_insert(c, p)`에 같은 c가 전달되므로 읽기·듣기 저장은 같은 트랜잭션입니다. 433~435행에서는 저장된 내용이 완전한지 확인합니다. 436행은 with 블록 밖이므로 정상 경로에서는 COMMIT 후 사후 조회가 실행됩니다.

문항과 문항 버전을 먼저 저장한다

코드 11 | 실제 코드 발췌 · _upsert_item_version() (중간 생략)

C:/Users/khyun/Projects/Unigate/topik_question_lab/postgres_storage.py

UI: 등록 내부 호출 / 입력: c, 문항 초안 / 출력: item_id, 버전, 생성 여부

```
368 def _upsert_item_version(connection, item: PublicationItemDraft, Jsonb) -> tuple[Any,
      int, bool]:
369     candidate = item.candidate
370     connection.execute(
371         """INSERT INTO topik_bank.items(item_id, source_key)
372            VALUES (%s, %s)
373            ON CONFLICT (source_key) DO NOTHING""",
374         (candidate.item_id, candidate.source_key),
375     )
...   # 중간 구간 생략
395     connection.execute(
396         """INSERT INTO topik_bank.item_versions(
397                item_id, item_version, section, type_slot, item_type, primary_skill,
398                target_level, predicted_difficulty, irt_difficulty, irt_discrimination,
399                stem_length, choice_count, generator_provider, generator_model,
400                generator_version, prompt_version, review_status, stem, choices,
401                correct_answer, explanation, content_json, source_provenance,
      content_hash
402            ) VALUES (
403                %s, %s, %s, %s, %s, %s, %s, %s, NULL, NULL, %s, %s, %s, %s,
404                %s, %s, 'reviewed', %s, %s, %s, %s, %s, %s, %s
405            )""",
...   # 중간 구간 생략
421             item.stem,
422             Jsonb(item.choices),
423             candidate.question.get("answer"),
424             str(candidate.question.get("explanation", "")),
425             Jsonb(candidate.question),
426             Jsonb(provenance),
427             item.content_hash,
428         ),
429     )
430     return item_id, item_version, True
```

370~375행은 문항 신원을 INSERT합니다. 376~394행(생략)은 문항 ID·기존 버전·다음 버전 번호와 출처를 준비합니다. 395~429행은 보기·정답·모델·전체 JSON과 출처를 버전 행에 저장합니다.

422·425·426행의 Jsonb는 6쪽의 변환과 같습니다. 직접 등록은 helper 호출 전에 check_remote()로 다른 내용을 충돌 처리하므로 기존 원본의 다른 내용을 자동 발행하지 않습니다.

세트 행과 1~50번 연결을 저장한다

코드 12 | 실제 코드 발췌 · DirectPublicationService._insert()

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 등록 내부 호출 / 입력: c, 한 세트 패키지 / 출력: 연결 행 저장

```
445 def _insert(self, c, package):
446     draft = package_draft(package)
447     _, Jsonb = _psycopg()
448     versions = []
449     for item in draft.items:
450         item_id, version, _ = _upsert_item_version(c, item, Jsonb)
451         versions.append((item_id, version))
452     identity = (draft.section, draft.backend, draft.provider_key, draft.model_id)
453     sequence = c.execute("""SELECT COALESCE(MAX(set_sequence),0)+1 FROM
    topik_bank.question_sets
454         WHERE section=%s AND generator_provider=%s AND generator_model=%s AND
    generator_version=%s""", identity).fetchone()[0]
455     c.execute("""INSERT INTO topik_bank.question_sets(set_id,section,generator_provider,
    generator_model,
456         generator_version,set_sequence,review_status,default_target_level,
    default_predicted_difficulty,set_fingerprint)
457         VALUES (%s,%s,%s,%s,%s,%s,'reviewed',%s,%s,%s)""",
458         (package["set_id"], *identity, sequence, package["level"],
    package["difficulty"],
459          set_fingerprint(versions, package["level"], package["difficulty"])))
460     _insert_set_items(c, [(package["set_id"], pos, item_id, version) for pos, (item_id,
    version) in enumerate(versions, 1)])
```

코드 13 | 실제 코드 발췌 · _insert_set_items()

C:/Users/khyun/Projects/Unigate/topik_question_lab/postgres_storage.py

UI: 등록 내부 호출 / 입력: 동일 연결·50개 튜플 / 출력: 50행 INSERT

```
451 def _insert_set_items(connection, rows: list[tuple]) -> None:
452     with connection.cursor() as cursor:
453         cursor.executemany(
454             """INSERT INTO topik_bank.question_set_items(
455                    set_id, position, item_id, item_version
456                ) VALUES (%s, %s, %s, %s)""",
457             rows,
458         )
```

versions에는 각 문항의 `(item_id, item_version)`이 모입니다. 같은 모델·영역의 순번을 계산하고, 결정적 set_id와 세트 해시로 세트 행을 저장합니다. enumerate(..., 1)가 1~50번을 연결합니다.

진행 중에는 누가 어떤 데이터를 볼까

현재 작업이 추가하는 문항 행만 계산

시점	같은 연결 A	다른 연결 B
등록 전	0	0
INSERT 후 / 커밋 전	100	0
성공: COMMIT 후 새 조회	100	100
실패: ROLLBACK 후	0	0

그림 7. 다른 연결 B는 각 시점에 새 SELECT를 실행한다. 새 문항 행만 계산하며 COMMIT과 ROLLBACK은 서로 다른 종료 경로다.

쓰기 연결 A는 자신이 넣은 미커밋 행을 볼 수 있습니다. 그래서 COMMIT 전에 `check_remote(c, p)`가 INSERT 결과를 확인할 수 있습니다. 일반적인 다른 연결 B는 A의 미커밋 행을 보지 못합니다.

현재 쓰기 연결은 기본 Read Committed를 사용합니다. 각 SQL은 시작 시점까지 커밋된 데이터를 기준으로 읽습니다. prepare()와 reconcile()의 조회 연결은 Repeatable Read로 여러 조회의 일관된 스냅샷을 사용합니다.

> 이미 Repeatable Read 스냅샷을 잡은 다른 연결이 있다면, COMMIT 이후에도 같은 트랜잭션에서는 이전 상태를 볼 수 있습니다. 위 그림의 "커밋 후 100"은 새 조회·새 스냅샷 기준입니다.

근거: PostgreSQL MVCC [9] · 격리 수준 [3]

중복 클릭과 불완전한 연결을 막는 장치

코드 14 | 실제 코드 발췌 · 트랜잭션 advisory lock

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 등록·재시도 / 같은 잠금 규칙을 쓰는 발행을 순서대로 실행

```
423 for lock in ("topik_bank:production_deployment", "topik_bank:set_membership",
      "topik_bank:item_sync"):
424     c.execute("SELECT pg_advisory_xact_lock(hashtext(%s))", (lock,))
425 require_target_schema(c)
426 # Lock acquisition may have waited while someone edited SQLite.
427 self._check_current(job)
428 packages = job["payload"]["packages"]
```

잠금 이름은 배포, 세트 연결, 문항 동기화 순서로 획득합니다. 같은 규칙을 사용하는 다른 작업은 기다립니다. `pg_advisory_xact_lock`은 트랜잭션이 끝나면 자동으로 해제됩니다.

장치	담당하는 안전성
Advisory lock	협력하는 발행·배포 작업의 동시 실행 조정
PRIMARY KEY / UNIQUE	동일 ID, source_key, 세트 번호·순서 중복 차단
FOREIGN KEY	존재하는 세트·문항 버전만 참조하도록 검사
validate_batch / check_remote	50문항, 승인, 형식, 내용·버전의 완전성 검사
SHA-256 / UUIDv5	내용 변경 판별 / 같은 묶음의 신원 유지

> Advisory lock은 모든 관리자 SQL을 강제로 잠그는 장치가 아닙니다. 같은 잠금을 쓰지 않는 외부 writer까지 직렬화하지 않습니다. 또한 DB 제약만으로 세트에 50행이 모두 있는지를 보장하지는 않으므로 앱 검증이 필요합니다.

근거: PostgreSQL advisory lock [10] · 현재 스키마 검사 코드

COMMIT은 PostgreSQL에서 무엇을 바꿀까



그림 8. 정상적으로 커밋된 원격 변경은 새로운 조회에서 사용할 수 있다.

ACID를 이번 문항 등록에 연결하기

성질	이번 사례에서의 뜻
Atomicity / 원자성	선택한 세트 저장이 함께 확정되거나 함께 취소됨
Consistency / 일관성	DB 제약과 앱 검증이 요구하는 관계·조건을 지킴
Isolation / 격리성	다른 조회는 미커밋 상태를 그대로 읽지 않음
Durability / 지속성	확정한 변경을 장애 이후에도 복구할 수 있도록 기록

PostgreSQL은 WAL(Write-Ahead Log)에 변경과 커밋 정보를 기록해 장애 복구를 지원합니다. 현재 확인한 `synchronous_commit=on` 설정에서 성공 응답은 WAL의 지속성 확보와 관련됩니다. 본 데이터 파일 전체를 그 순간 즉시 다시 쓰는 것과는 다릅니다.

WAL은 데이터 파일보다 로그를 먼저 안전하게 기록하는 원칙입니다. 데이터 파일 반영은 이후에 진행될 수 있고, 장애 후에는 필요한 로그를 다시 적용해 확정된 상태를 복구합니다.

> COMMIT은 원격 DB의 확정 시점입니다. 로컬 앱이 성공 응답을 받았는지, 이후 결과 기록까지 끝냈는지는 별개의 문제입니다. 이 차이가 22~23쪽 복구 과정의 출발점입니다.

근거: PostgreSQL WAL [11] · 트랜잭션 기본 [12]

커밋 후 새 연결에서 다시 확인한다

코드 15 | 실제 코드 발췌 · DirectPublicationService.reconcile()

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 원격 결과 다시 확인·등록 직후 / 입력: plan_id / 출력: 원격 대조 결과와 로컬 상태

```
391 def reconcile(self, plan_id):
392     job = self._job(plan_id)
393     try:
394         with self.connect(readonly=True) as c:
395             c.execute("SET TRANSACTION ISOLATION LEVEL REPEATABLE READ")
396             require_target_schema(c)
397             results = [check_remote(c, p) for p in job["payload"]["packages"]]
398         status = "succeeded" if all(r["status"] == "complete" for r in results) else
    "failed"
399         self.jobs.update(plan_id, status, {"sets": results})
400     except PublicationBlocked as exc:
401         self.jobs.update(plan_id, "conflict", message=_safe_error(exc, self.target_url))
402     except Exception as exc:
403         self.jobs.update(plan_id, "outcome_unknown", message=_safe_error(exc,
    self.target_url))
404     return self.jobs.get(plan_id)
```

check_remote()의 검사	확인하는 값
문항	item_id, source_key, 내용·해시, 출처
세트	영역·모델·검수 상태·급수·난이도
연결	번호 1~50, 정확한 item_id와 item_version
해시	세트가 참조하는 버전 목록의 fingerprint
서비스 조회 뷰	current_set_contents의 50문항 내용

검사 결과가 모두 complete이면 로컬 작업을 succeeded로 기록합니다. 원격 내용이 충돌하면 conflict, 연결을 못 해 결과를 판단할 수 없으면 outcome_unknown입니다. 일부 또는 미반영 상태는 failed로 기록하고 재시도 경로에서 다시 판단합니다.

두 번째 세트가 실패하면 첫 번째도 취소된다



그림 9. 읽기 INSERT가 성공했어도 COMMIT 전이므로 전체 트랜잭션을 취소할 수 있다.

관찰 대상	실패 전	ROLLBACK 후
현재 작업의 원격 추가 행	일부를 A 연결에서 볼 수 있음	이번 트랜잭션의 추가 행 0
원격의 기존 문항·세트	이미 존재함	기존 상태 유지
로컬 생성·편집 문항	SQLite에 존재함	원본 유지
로컬 작업 기록	publishing	재조회에 따라 failed / 결과 불명

이는 "먼저 올라간 문제를 삭제하는 보상 작업"이 아니라, 아직 확정하지 않은 한 트랜잭션을 취소하는 것입니다. 단, 이미 존재하던 같은 데이터의 재사용은 이번 트랜잭션이 생성한 행이 아니므로 없어지지 않습니다.

통신이 끊겨도 곧바로 실패는 아니다



그림 10. 서버가 확정한 시점과 앱이 성공 응답을 받은 시점은 다르다.

코드 16 | 실제 코드 발췌 · publish()의 예외 후 원격 재확인

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 등록 오류·같은 작업 재시도 / 입력: 예외 / 출력: 실제 반영 결과

```
436         return self.reconcile(plan_id)
437     except Exception as exc:
438         message = _safe_error(exc, self.target_url)
439         result = self.reconcile(plan_id)
440         if result["status"] != "succeeded":
441             self.jobs.update(plan_id, result["status"], result["result"], message)
442             raise PublicationBlocked(message) from exc
443         return result
```

예외 처리에서도 reconcile()을 호출합니다. 실제 원격 데이터가 정확하게 존재하면 성공 결과를 돌려줍니다. 원격에 접속하지 못하면 결과 불명으로 남기고, 연결을 복구한 뒤 같은 작업으로 다시 확인합니다.

> 확실히 미반영인지 판단하지 못한 상황에서 새 세트 ID로 다시 등록하면 중복이 생길 수 있습니다. 고정 작업·고정 세트 ID와 원격 대조가 이 문제를 줄입니다.

로컬 기록과 원격 저장은 별도 트랜잭션



같은 시점에도 두 DB의 상태는 다를 수 있습니다.

그림 11. 윗줄은 로컬 작업 상태, 아랫줄은 원격 저장 상태다. 두 DB는 별도 트랜잭션으로 변한다.

이 앱은 SQLite와 원격 PostgreSQL을 하나의 분산 트랜잭션으로 묶지 않습니다. 원격 저장의 원자성은 PostgreSQL 안에서 보장하고, 로컬 작업 기록의 지연·실패는 원격 내용을 다시 조회해서 복구합니다.

코드 17 | 실제 코드 발췌 · 서로 다른 DB에 기록되는 지점

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 등록 시작·등록 결과 / 로컬 UPDATE와 원격 with 블록을 구분

```
173  def update(self, plan_id, status, result=None, message=""):
174      with self.connect() as c:
175          c.execute("UPDATE jobs SET status=?,result=?,message=?,
       updated_at=CURRENT_TIMESTAMP WHERE id=?",
176                    (status, _json(result or {}), message, plan_id))
     # 중간 구간 생략
419      self.jobs.update(plan_id, "publishing")
420      try:
421          with self.connect() as c:
     # 중간 구간 생략
398          status = "succeeded" if all(r["status"] == "complete" for r in results) else
       "failed"
399          self.jobs.update(plan_id, status, {"sets": results})
```

> "등록 완료"를 표시하려면 원격 저장뿐 아니라 사후 검증과 로컬 결과 기록도 끝나야 합니다. 이미 저장됐는데 앱이 성공을 표시하지 못하는 경우를 이 구조로 설명할 수 있습니다.

로컬 DB 쓰기 장애가 계속되면 상태 갱신도 실패할 수 있습니다. 로컬 저장소를 복구한 뒤 "원격 결과 다시 확인"으로 이미 커밋된 작업을 대조하고 결과를 기록합니다.

재시도는 무엇을 다시 하는가

확인 결과	앱의 처리	사용자가 보는 의미
모든 내용·버전이 일치	existing complete를 재사용	등록 완료, INSERT 반복 없음
세트가 아직 없음	원격 잠금 안에서 다시 검사 후 INSERT	같은 작업 재시도 가능
내용·메타데이터·연결 충돌	자동 덮어쓰기 차단	충돌 원인 확인 필요
원격 접속 불가	쓰기 결과를 단정하지 않음	결과 확인 필요
검증 후 로컬 원본 변경	기존 고정 작업으로 등록 차단	재검증 필요

코드 18 | 실제 코드 발췌 · 재시도 전에 reconcile()

C:/Users/khyun/Projects/Unigate/topik_question_lab/direct_publication.py

UI: 같은 작업 재시도 / 기존 시도의 실제 원격 상태부터 확인

```
413 if job["status"] in {"publishing", "outcome_unknown", "succeeded"}:
414     checked = self.reconcile(plan_id)
415     if checked["status"] == "succeeded":
416         return checked
417     if checked["status"] != "failed":
418         raise PublicationBlocked(checked["message"] or "원격 반영 결과를 먼저 확인해야
    합니다.")
```

다시 클릭했다고 항상 INSERT를 하는 것은 아닙니다. 원격 잠금과 내용 비교를 통과한 미등록 세트만 저장합니다. 이미 동일하게 반영된 세트는 재사용하므로 같은 작업을 반복해도 세트가 계속 늘지 않습니다.

등록된 묶음을 로컬에서 수정하면 목록은 "등록 후 변경됨"으로 표시합니다. 이 흐름은 이미 발행한 원격 문제를 자동 수정하는 기능이 아니며, 다른 내용은 충돌로 차단합니다.

실제 사례: 읽기·듣기 100문항 등록

> 2026-10-04 실제 실행 기록: 두 세트의 원격 등록·사후 조회가 성공했습니다. 문항 은행 등록과 모의고사 공개는 별개이며, 음원·그림 생성과 학습자 공개는 이 작업에서 수행하지 않았습니다.

테이블	등록 전: 이번 묶음의 행	등록 후: 이번 작업 추가 행
items	0	100
item_versions	0	100
question_sets	0	2
question_set_items	0	100

위 표는 이번 작업의 증가량입니다. 기존 원격 DB에는 다른 문항과 세트가 있었고, 등록 전후 대조에서 기존 행이 보존됐음을 확인했습니다. 테이블·열·제약 구조도 변경하지 않았습니다.

영역	원격 set_id	확인 결과
읽기	6f8a0aac-546b-56a6-8b32-784620eb656e	1~50번 / 유효 50개
듣기	98928e82-965f-5c19-b01c-90a67f40a1c0	1~50번 / 유효 50개

읽기 batch_id: `95e2f11a2d5e4d1aa033c34cdb167855`
듣기 batch_id: `a379d1e1aaef4f4c9244c7ed19994814`

고정 작업 ID:
`bc2cb9a38cdbd9e2bc1aec811d722486e85b7ace2e1eec1de5defdd6964283d4`

근거 위치: C:/Users/khyun/Projects/Unigate/POSTGRESQL_QUESTION_BANK.md
로컬 작업 DB: C:/Users/khyun/Projects/Unigate/data/publication_jobs.db

어느 파일을 열면 무엇을 볼 수 있을까

아래 앱 파일은 모두 다음 폴더 안에 있습니다:
C:/Users/khyun/Projects/Unigate/topik_question_lab

파일 / 시작 위치	담당 역할
main.py:14	Streamlit 메뉴와 페이지 진입점
bulk_generation_app.py:128	생성 완료 후 세트·원격 등록 바로가기
bulk_generation.py:123	생성 묶음·유형별 작업 기록과 실행
generation_service.py:70	공통 지문·대본과 슬롯 구조 검사
generation_storage.py:39	생성 결과와 batch_id 연결을 원본 SQLite에 저장
direct_publish_app.py:68, 82, 109	검증·등록·결과 확인 UI와 서비스 호출
direct_publication.py:354, 406, 391	prepare / publish / reconcile 전체 흐름
publication.py:151, 209, 238	원본 후보 수집과 세트·문항 초안 변환
postgres_storage.py:368, 451	문항 버전 저장과 50개 연결 행 INSERT
models.py:57 / listening_models.py:97	Pydantic 문항 필드와 자료형
validation.py:58 / listening_validation.py:17	현재 문제의 형식·내용 구조 검사
backup_service.py:107, 190	로컬 발행 작업 DB를 포함한 백업·복원

빠른 코드 이동: UI는 8쪽, prepare는 9쪽, connect는 11쪽, publish는 13~14쪽, INSERT는 15~16쪽, reconcile은 20쪽입니다. 코드 블록의 줄 번호는 제작 시점의 원본 번호입니다. 이후 파일을 수정하면 달라질 수 있습니다.

용어를 다시 확인하고 오해를 정리하기

용어	이 가이드에서의 뜻
BEGIN / COMMIT / ROLLBACK	시작 / 변경 확정 / 미확정 변경 취소
autocommit=False	각 SQL을 즉시 독립 확정하지 않고 트랜잭션으로 실행
MVCC / snapshot	여러 버전·조회 시점 기준으로 데이터 가시성을 관리
Read Committed	각 SQL 시작 전까지 커밋된 데이터 기준으로 조회
Repeatable Read	한 조회 트랜잭션에서 일관된 스냅샷 사용
advisory lock	같은 규칙을 따르는 작업끼리 사용하는 협력적 잠금
WAL	장애 복구·지속성을 위한 선행 기록
JSONB	PostgreSQL의 JSON 자료형
idempotent / 멱등	같은 작업 반복이 중복 등록으로 이어지지 않는 성질
reconcile	로컬 작업 내용과 실제 원격 반영 결과의 대조

네 가지 문장을 기억하세요

① "SQL이 성공했다"는 "COMMIT까지 끝났다"와 다릅니다.
② "등록 완료"는 "학생에게 공개됐다"와 다릅니다.
③ "네트워크 오류"만으로 "원격 저장 실패"를 단정할 수 없습니다.
④ 성공한 COMMIT을 이후 ROLLBACK으로 되돌릴 수는 없습니다.

트랜잭션은 선택한 원격 변경을 함께 확정하는 단위입니다. UI의 버튼 하나, SQL 하나, 네트워크 요청 하나, 로컬 작업 기록 하나와 같은 개념은 아닙니다.

공식 문서와 코드 근거

[1] Psycopg - Transactions management

[2] Psycopg - Basic module usage / Connection context

[3] PostgreSQL 17 - Transaction Isolation

[4] Supabase - Connect to your database

[5] Psycopg - pq / libpq wrapper

[6] Psycopg - Passing parameters to SQL queries

[7] Psycopg - Adapting basic Python types / JSON

[8] Psycopg - Prepared statements

[9] PostgreSQL 17 - MVCC introduction

[10] PostgreSQL 17 - Explicit / advisory locking

[11] PostgreSQL 17 - Write-Ahead Logging

[12] PostgreSQL 17 - Transactions tutorial

실제 구현과 문서의 관계

공식 문서는 기술의 일반 원리를 뒷받침합니다. 이 앱의 함수 순서, UI 상태, 저장 필드와 제한은 현재 저장소 코드를 근거로 설명했습니다. 실제 코드와 학습 예제를 혼동하지 않도록 각각 표시했습니다.

환경 정보는 2026-10-05에 읽기 전용으로 확인한 연결 모드·서버 설정과 프로젝트의 설치 버전 기준입니다. 이번 PDF 제작은 앱 코드나 원격 데이터를 변경하지 않습니다.

본문 코드는 원래 줄 번호를 유지하고 긴 줄만 지면에서 이어 표시했습니다. 접속 URL·호스트·사용자·비밀번호·API 키는 포함하지 않았습니다.

> 다시 볼 핵심 경로: UI → prepare → 로컬 고정 작업 → publish → 원격 COMMIT → reconcile → 등록 완료.